

OAuth Token Leak via Referer Policy Override Demo

OAuth Token Leak via Referer Policy Override Demo - Amirmohammad Safari, Voorivex.

- Published: date not stated
- Original: <<https://github.com/VoorivexTeam/white-box-challenges/tree/bfda4647384e6590c9da4b76d21da3cc2eda638e/referer-override>>
- Preserved from: <https://github.com/VoorivexTeam/white-box-challenges/tree/bfda4647384e6590c9da4b76d21da3cc2eda638e/referer-override> (manual-import) on 2026-09-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

OAuth Token Leak via Referer Policy Override Demo

Authors: Amirmohammad Safari

Source: <https://github.com/VoorivexTeam/white-box-challenges/tree/bfda4647384e6590c9da4b76d21da3cc2eda638e/referer-override>

Preservation note: These are the complete text contents of the reviewed files from this pinned revision. File contents were checked against their Git blob hashes. This document preserves source text for reading; it is not a runnable repository copy.

referer-override/README.md

OAuth Token Leak via Referer Policy Override Demo

[image: banner.jpg]

Setup

1. Add to `/etc/hosts` :

```
127.0.0.1    company.tld
127.0.0.1    sso.company.tld
```

1. Install dependencies for SSO and company:

```
npm install
```

1. Start the server:

```
node index.js
```

Access the application at `http://company.tld:3001`

referer-override/company.tld/index.js

```
const express = require('express');
const { AuthorizationCode } = require('simple-oauth2');
const axios = require('axios');
const path = require('path');
const session = require('express-session');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.use(session({
  secret: 'supersecretkey',
  resave: false,
  saveUninitialized: false,
  cookie: { secure: false }
}));

const config = {
  client: {
    id: 'client1',
    secret: 'secret'
  },
  auth: {
    tokenHost: 'http://sso.company.tld:3000',
    tokenPath: '/token',
    authorizePath: '/authorize'
  }
};

const oauth2Client = new AuthorizationCode(config);

const isAuthenticated = (req, res, next) => {
  if (req.session.user) {
    return next();
  }
  res.redirect('/');
};

app.get('/', (req, res) => {
  if (req.session.user) {
    return res.redirect('/profile');
  }
});
```

```
const authorizationUri = oauth2Client.authorizeURL({
  redirect_uri: 'http://company.tld:3001/callback',
  response_type: 'code',
  state: 'random-state',
  scope: 'openid profile email'
});

res.render('home', { authorizationUri });
});

app.get('/callback', async (req, res) => {
  const { code } = req.query;

  if (!code) {
    return res.redirect('/');
  }

  try {
    const tokenParams = {
      code,
      redirect_uri: 'http://company.tld:3001/callback',
      scope: 'openid profile'
    };

    const result = await oauth2Client.getToken(tokenParams);
    const { token } = result;

    req.session.token = token;
    req.session.user = {
      id: token.user.id,
      username: token.user.username,
      email: token.user.email,
      firstName: token.user.firstName,
      lastName: token.user.lastName
    };

    res.redirect('/profile');
  } catch (error) {
    console.error('Error getting token:', error.message);
    res.render('error', {
      message: 'Authentication failed: ' + error.message
    });
  }
});
```

```
app.get('/profile', isAuthenticated, (req, res) => {  
  res.render('profile', { user: req.session.user });  
});  
  
app.get('/dompurify', (req, res) => {  
  res.render('dompurify');  
});  
  
app.get('/logout', (req, res) => {  
  req.session.destroy();  
  res.redirect('/');  
});  
  
app.listen(3001, () => {  
  console.log('OAuth2 Client server listening on port 3001');  
});
```

referer-override/company.tld/views/dompurify.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sanitized HTML Render</title>
  <script src="https://cdn.jsdelivr.net/npm/dompurify@3.2.5/dist/purify.min.js"></script>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
  <div class="container mt-5">
    <div class="card shadow-sm mx-auto" style="max-width: 600px;">
      <div class="card-body">
        <h1 class="h4 mb-4">Sanitized HTML Output</h1>
        <div id="sanitized-output" class="border p-3 bg-white"></div>
      </div>
    </div>
  </div>

  <script>
    // Extract query parameter from the URL
    const params = new URLSearchParams(window.location.search);
    const rawInput = params.get('input');

    if (rawInput) {
      const cleanHTML = DOMPurify.sanitize(rawInput);
      document.getElementById('sanitized-output').innerHTML = cleanHTML;
    } else {
      document.getElementById('sanitized-output').textContent = "No input provided.";
    }
  </script>
</body>
</html>
```

referer-override/sso.company.tld/index.js

```
const express = require('express');
const OAuth2Server = require('./oauth2-server');
const bodyParser = require('body-parser');
const path = require('path');

const app = express();

// Setup view engine
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Parse application/x-www-form-urlencoded
app.use(bodyParser.urlencoded({ extended: false }));

// Parse application/json
app.use(bodyParser.json());

// In-memory data store
const users = [{
  id: '1',
  username: 'user',
  firstName: 'John',
  lastName: 'Doe',
  email: 'user@voorivex.team',
  password: 'password'
}];

const clients = [{
  id: 'client1',
  clientSecret: 'secret',
  redirectUris: ['http://company.tld:3001/callback'],
  grants: ['authorization_code', 'refresh_token']
}];

const authorizationCodes = [];
const tokens = [];

// OAuth2 model implementation
const model = {
  // Get client for authentication
  getClient: (clientId, clientSecret) => {
    console.log(clientId, clientSecret);
    const client = clients.find(client => client.id === clientId);
    if (!client || (clientSecret && client.clientSecret !== clientSecret)) {
      return false;
    }
  }
}
```

```
}
  return client;
},

// Save authorization code
saveAuthorizationCode: (code, client, user) => {
  const authCode = {
    authorizationCode: code.authorizationCode,
    expiresAt: code.expiresAt,
    redirectUri: code.redirectUri,
    client: client,
    user: user
  };
  authorizationCodes.push(authCode);
  return authCode;
},

// Get authorization code
getAuthorizationCode: (authorizationCode) => {
  return authorizationCodes.find(code =>
    code.authorizationCode === authorizationCode
  );
},

// Revoke authorization code
revokeAuthorizationCode: (code) => {
  const index = authorizationCodes.findIndex(c =>
    c.authorizationCode === code.authorizationCode
  );
  if (index !== -1) {
    authorizationCodes.splice(index, 1);
    return true;
  }
  return false;
},

// Save token
saveToken: (token, client, user) => {
  const accessToken = {
    accessToken: token.accessToken,
    accessTokenExpiresAt: token.accessTokenExpiresAt,
    refreshToken: token.refreshToken,
    refreshTokenExpiresAt: token.refreshTokenExpiresAt,
    client: client,
  };
}
```

```
    user: user
  };
  tokens.push(accessToken);
  return accessToken;
},

// Get access token
getAccessToken: (accessToken) => {
  return tokens.find(token => token.accessToken === accessToken);
},

// Get refresh token
getRefreshToken: (refreshToken) => {
  return tokens.find(token => token.refreshToken === refreshToken);
},

// Revoke token
revokeToken: (token) => {
  const index = tokens.findIndex(t => t.refreshToken === token.refreshToken);
  if (index !== -1) {
    tokens.splice(index, 1);
    return true;
  }
  return false;
},

// Get user by username and password
getUser: (username, password) => {
  const user = users.find(user =>
    user.username === username && user.password === password
  );
  return user || false;
},

// Validate scope
validateScope: (user, client, scope) => {
  return user.scope === client.scope
},

// Verify scope
verifyScope: (token, scope) => {
  if (!token.scope) {
    return false;
  }
}
```

```

    const requestedScopes = scope.split(' ');
    const authorizedScopes = token.scope.split(' ');
    return requestedScopes.every(s => authorizedScopes.includes(s));
  }
};

// Initialize OAuth server
const oauth = new OAuth2Server({
  model: model,
  accessTokenLifetime: 60 * 60, // 1 hour
  allowBearerTokensInQueryString: true
});

// GET /authorize - Show login form
app.get('/authorize', async (req, res) => {
  res.render('login', {
    client_id: req.query.client_id,
    redirect_uri: req.query.redirect_uri,
    response_type: req.query.response_type,
    state: req.query.state || '',
    scope: req.query.scope || '',
    error: null
  });
});

// POST /authorize - Handle login form submission
app.post('/authorize', async (req, res) => {
  const { username, password, client_id, redirect_uri, response_type, state, scope } = req.body;

  try {
    // Authenticate user
    const user = await model.getUser(username, password);
    if (!user) {
      return res.render('login', {
        client_id,
        redirect_uri,
        response_type,
        state,
        scope,
        error: 'Invalid username or password'
      });
    }
  }
});

// Create request and response objects for OAuth server

```

```
const request = new OAuth2Server.Request({
  method: 'POST',
  query: {
    response_type,
    client_id,
    redirect_uri,
    state,
    scope
  },
  headers: req.headers,
  body: { username, password }
});
```

```
const response = new OAuth2Server.Response(res);
```

```
// Authorize
```

```
await oauth.authorize(request, response, {
  authenticateHandler: {
    handle: () => user
  }
});
```

```
// Redirect handled by OAuth2 server
```

```
return res.set(response.headers).status(response.status).send(response.body);
} catch (error) {
  console.error('OAuth error:', error);
  return res.render('login', {
    client_id,
    redirect_uri,
    response_type,
    state,
    scope,
    error: error.message || 'An error occurred during authorization'
  });
}
});
```

```
// Token endpoint
```

```
app.post('/token', async (req, res) => {
  try {
    const request = new OAuth2Server.Request(req);
    const response = new OAuth2Server.Response(res);

    const token = await oauth.token(request, response);
```

```
    return res.set(response.headers).status(response.status).json(token);
  } catch (error) {
    console.error('Token error:', error);
    return res.status(error.code || 500).json(error);
  }
});

// Start the provider server
app.listen(3000, () => {
  console.log('OAuth2 Provider server listening on port 3000');
});
```

referrer-override/sso.company.tld/oauth2-server/lib/handlers/authorize-handler.js

```

'use strict';

/**
 * Module dependencies.
 */

var _ = require('lodash');
var AccessDeniedError = require('../errors/access-denied-error');
var AuthenticateHandler = require('../handlers/authenticate-handler');
var InvalidArgumentError = require('../errors/invalid-argument-error');
var InvalidClientError = require('../errors/invalid-client-error');
var InvalidRequestError = require('../errors/invalid-request-error');
var InvalidScopeError = require('../errors/invalid-scope-error');
var UnsupportedResponseTypeError = require('../errors/unsupported-response-type-error');
var OAuthError = require('../errors/oauth-error');
var Promise = require('bluebird');
var promisify = require('promisify-any').use(Promise);
var Request = require('../request');
var Response = require('../response');
var ServerError = require('../errors/server-error');
var UnauthorizedClientError = require('../errors/unauthorized-client-error');
var is = require('../validator/is');
var tokenUtil = require('../utils/token-util');
var url = require('url');

/**
 * Response types.
 */

var responseTypes = {
  code: require('../response-types/code-response-type'),
  //token: require('../response-types/token-response-type')
};

/**
 * Constructor.
 */

function AuthorizeHandler(options) {
  options = options || {};

  if (options.authenticateHandler && !options.authenticateHandler.handle) {
    throw new InvalidArgumentError('Invalid argument: authenticateHandler does not implement `handle()`');
  }
}

```

```

if (!options.authorizationCodeLifetime) {
  throw new InvalidArgumentError('Missing parameter: `authorizationCodeLifetime`');
}

if (!options.model) {
  throw new InvalidArgumentError('Missing parameter: `model`');
}

if (!options.model.getClient) {
  throw new InvalidArgumentError('Invalid argument: model does not implement `getClient()`');
}

if (!options.model.saveAuthorizationCode) {
  throw new InvalidArgumentError('Invalid argument: model does not implement `saveAuthorizationCode()`');
}

this.allowEmptyState = options.allowEmptyState;
this.authenticateHandler = options.authenticateHandler || new AuthenticateHandler(options);
this.authorizationCodeLifetime = options.authorizationCodeLifetime;
this.model = options.model;
}

/**
 * Authorize Handler.
 */

AuthorizeHandler.prototype.handle = function(request, response) {
  if (!(request instanceof Request)) {
    throw new InvalidArgumentError('Invalid argument: `request` must be an instance of Request');
  }

  if (!(response instanceof Response)) {
    throw new InvalidArgumentError('Invalid argument: `response` must be an instance of Response');
  }

  if ('false' === request.query.allowed) {
    return Promise.reject(new AccessDeniedError('Access denied: user denied access to application'));
  }

  var fns = [
    this.getAuthorizationCodeLifetime(),
    this.getClient(request),
    this.getUser(request, response)
  ];

```

```

];

return Promise.all(fns)
  .bind(this)
  .spread(function(expiresAt, client, user) {
    var uri = this.getRedirectUri(request, client);
    var scope;
    var state;
    var responseType;

    return Promise.bind(this)
      .then(function() {
        var requestedScope = this.getScope(request);

        return this.validateScope(user, client, requestedScope);
      })
      .then(function(validScope) {
        scope = validScope;

        return this.generateAuthorizationCode(client, user, scope);
      })
      .then(function(authorizationCode) {
        state = this.getState(request);
        responseType = this.getResponseType(request);

        return this.saveAuthorizationCode(authorizationCode, expiresAt, scope, client, uri, user);
      })
      .then(function(code) {
        var responseType = new ResponseType(code.authorizationCode);
        var redirectUri = this.buildSuccessRedirectUri(uri, responseType);

        this.updateResponse(response, redirectUri, state);

        return code;
      })
      .catch(function(e) {
        if (!(e instanceof OAuthError)) {
          e = new ServerError(e);
        }
        var redirectUri = this.buildErrorRedirectUri(uri, e);

        this.updateResponse(response, redirectUri, state);

        throw e;
      });

```

```

    });
  });
};

/**
 * Generate authorization code.
 */

AuthorizeHandler.prototype.generateAuthorizationCode = function(client, user, scope) {
  if (this.model.generateAuthorizationCode) {
    return promisify(this.model.generateAuthorizationCode, 3).call(this.model, client, user, scope);
  }
  return tokenUtil.generateRandomToken();
};

/**
 * Get authorization code lifetime.
 */

AuthorizeHandler.prototype.getAuthorizationCodeLifetime = function() {
  var expires = new Date();

  expires.setSeconds(expires.getSeconds() + this.authorizationCodeLifetime);
  return expires;
};

/**
 * Get the client from the model.
 */

AuthorizeHandler.prototype.getClient = function(request) {
  var clientId = request.body.client_id || request.query.client_id;

  if (!clientId) {
    throw new InvalidRequestError('Missing parameter: `client_id`');
  }

  if (!is.vschar(clientId)) {
    throw new InvalidRequestError('Invalid parameter: `client_id`');
  }

  var redirectUri = request.body.redirect_uri || request.query.redirect_uri;

  if (redirectUri && !is.uri(redirectUri)) {

```

```

    throw new InvalidRequestError('Invalid request: `redirect_uri` is not a valid URI');
  }
  return promisify(this.model.getClient, 2).call(this.model, clientId, null)
    .then(function(client) {
      if (!client) {
        throw new InvalidClientError('Invalid client: client credentials are invalid');
      }

      if (!client.grants) {
        throw new InvalidClientError('Invalid client: missing client `grants`');
      }

      if (!_includes(client.grants, 'authorization_code')) {
        throw new UnauthorizedClientError('Unauthorized client: `grant_type` is invalid');
      }

      if (!client.redirectUri || 0 === client.redirectUri.length) {
        throw new InvalidClientError('Invalid client: missing client `redirectUri`');
      }

      if (redirectUri) {
        const redirectUriOrigin = new URL(redirectUri).origin;
        const hasMatchingOrigin = client.redirectUri.some(uri => {
          try {
            return new URL(uri).origin === redirectUriOrigin;
          } catch (e) {
            return false;
          }
        });
      }

      if (!hasMatchingOrigin) {
        throw new InvalidClientError('Invalid client: `redirect_uri` does not match client value');
      }
    }
    return client;
  });
};

/**
 * Validate requested scope.
 */
AuthorizeHandler.prototype.validateScope = function(user, client, scope) {
  if (this.model.validateScope) {
    return promisify(this.model.validateScope, 3).call(this.model, user, client, scope)
  }

```

```

.then(function (scope) {
  if (!scope) {
    throw new InvalidScopeError('Invalid scope: Requested scope is invalid');
  }

  return scope;
});
} else {
  return Promise.resolve(scope);
}
};

```

```

/**
 * Get scope from the request.
 */

```

```

AuthorizeHandler.prototype.getScope = function(request) {
  var scope = request.body.scope || request.query.scope;

  if (!is.nqschar(scope)) {
    throw new InvalidScopeError('Invalid parameter: `scope`');
  }

  return scope;
};

```

```

/**
 * Get state from the request.
 */

```

```

AuthorizeHandler.prototype.getState = function(request) {
  var state = request.body.state || request.query.state;

  if (!this.allowEmptyState && !state) {
    throw new InvalidRequestError('Missing parameter: `state`');
  }

  if (!is.vschar(state)) {
    throw new InvalidRequestError('Invalid parameter: `state`');
  }

  return state;
};

```

```
/**
 * Get user by calling the authenticate middleware.
 */
```

```
AuthorizeHandler.prototype.getUser = function(request, response) {
  if (this.authenticateHandler instanceof AuthenticateHandler) {
    return this.authenticateHandler.handle(request, response).get('user');
  }
  return promisify(this.authenticateHandler.handle, 2)(request, response).then(function(user) {
    if (!user) {
      throw new ServerError('Server error: `handle()` did not return a `user` object');
    }

    return user;
  });
};
```

```
/**
 * Get redirect URI.
 */
```

```
AuthorizeHandler.prototype.getRedirectUri = function(request, client) {
  return request.body.redirect_uri || request.query.redirect_uri || client.redirectUris[0];
};
```

```
/**
 * Save authorization code.
 */
```

```
AuthorizeHandler.prototype.saveAuthorizationCode = function(authorizationCode, expiresAt, scope, client, redirectUri) {
  var code = {
    authorizationCode: authorizationCode,
    expiresAt: expiresAt,
    redirectUri: redirectUri,
    scope: scope
  };
  return promisify(this.model.saveAuthorizationCode, 3).call(this.model, code, client, user);
};
```

```
/**
 * Get response type.
 */
```

```
AuthorizeHandler.prototype.getResponseTypes = function(request) {
```

```

var responseType = request.body.response_type || request.query.response_type;

if (!responseType) {
  throw new InvalidRequestError('Missing parameter: `response_type`');
}

if (!_has(responseTypes, responseType)) {
  throw new UnsupportedResponseTypeError('Unsupported response type: `response_type` is not supported');
}

return responseTypes[responseType];
};

/**
 * Build a successful response that redirects the user-agent to the client-provided url.
 */

AuthorizeHandler.prototype.buildSuccessRedirectUri = function(redirectUri, responseType) {
  return responseType.buildRedirectUri(redirectUri);
};

/**
 * Build an error response that redirects the user-agent to the client-provided url.
 */

AuthorizeHandler.prototype.buildErrorRedirectUri = function(redirectUri, error) {
  var uri = url.parse(redirectUri);

  uri.query = {
    error: error.name
  };

  if (error.message) {
    uri.query.error_description = error.message;
  }

  return uri;
};

/**
 * Update response with the redirect uri and the state parameter, if available.
 */

AuthorizeHandler.prototype.updateResponse = function(response, redirectUri, state) {

```

```
redirectUri.query = redirectUri.query || {};
```

```
if (state) {  
  redirectUri.query.state = state;  
}
```

```
response.redirect(url.format(redirectUri));  
};
```

```
/**  
 * Export constructor.  
 */
```

```
module.exports = AuthorizeHandler;
```

Archived from <https://github.com/VoorivexTeam/white-box-challenges/tree/bfda4647384e6590c9da4b76d21da3cc2eda638e/referer-override>. Rendered offline from the local Markdown copy.